

Ops Window — Role Briefs (Research & Development)

Sep 27, 2026 · @Tasnim

Briefs for Persons 1, 2, 4 and 5 on the NASA Space Apps 2026 entry. Persons 1 and 2 have deadlines before the hackathon; Persons 4 and 5 have one small pre-event task each and then build on the weekend.

What we're building

Two angles decide what a lunar lander can do: how high the Sun sits above the horizon (power) and how high Earth sits above the horizon (communications). Neither alone is useful. A lander with power and no link is deaf; a lander with a link and no power is dead. Real work only happens when both are up at once.

That overlap is the **ops window**, and nobody visualises it. Mission pages list a landing date and a payload manifest. They do not tell you that one mission had six days of usable time and another had two weeks, or that a far-side lander has zero direct-to-Earth time by geometry alone.

We compute that overlap for every lunar landing site we can get coordinates for, from Surveyor to the CLPS landers still to fly, and we show it. The historical missions are not filler. Apollo sites sit near-side and equatorial, so Earth is permanently up and geometry was never the constraint. Chang'e 4 sits on the far side, so direct-to-Earth time is exactly zero, which is why the Queqiao relay flew first. CLPS is now pushing to 80–85° south, where both bodies skim the horizon. Same calculation, three different stories, and the old missions are the control group that makes the polar difficulty legible.

The deliverable is a single-page web app with no backend: a lunar map, a timeline, a polar sky plot, a comparison chart and a mission detail panel, all reading one precomputed JSON file. It works offline because of that.

Person 1 — Mission research

Your job in one line: produce a filled-in spreadsheet of 22 lunar landing sites with verified coordinates and mission facts. Everything else in the project is computed from your table. A wrong coordinate gives a wrong answer with no error message.

What to produce

One spreadsheet, one row per mission, these columns:

Column	Notes
<code>mission_id</code>	lowercase slug, e.g. <code>apollo-11</code> , <code>im-2</code>
<code>mission</code>	display name
<code>provider</code>	NASA, CNSA, ISRO, Intuitive Machines...
<code>site_name</code>	Mare Crisium, Malapert A...
<code>lat_deg</code>	+North, decimal degrees
<code>lon_east_deg</code>	+East , decimal degrees
<code>landing_utc</code>	date and time if known
<code>status</code>	success / partial / failed / planned
<code>ops_duration</code>	how long it actually operated
<code>end_reason</code>	why it stopped
<code>payloads</code>	short list
<code>source_url</code>	where the coordinate came from
<code>confidence</code>	confirmed / verify

The one trap

The `lon_east_deg` column is where this breaks. Many sources write longitude west-positive. If a source says 23.4° W, you enter **-23.4**. If it says 316.6° E, leave it, that is the same place. Get the sign wrong and the mission lands on the wrong side of the Moon. Person 2's sanity check will catch it, but check it yourself first.

Where to get it

Apollo — use this and nothing else: [NSSDCA Apollo landing site coordinates](#). All six sites to five decimal places, derived from LRO imagery, already east-positive. Use the Lunar Module row, not LRRR or ALSEP, since the lander is what we model.

Everything else robotic: [NSSDCA table of spacecraft on the Moon](#). Every human object on the lunar surface, covering Luna, Surveyor, Lunokhod and later arrivals. It gives both the W-negative and E-positive forms side by side, so take the second and the sign cannot go wrong. It also marks which sites LRO has imaged, which is your confidence flag.

Recent missions (Chang'e, Chandrayaan-3, SLIM, Odysseus, Blue Ghost, IM-2): [LROC](#), search the mission name. LROC gives authoritative post-landing positions, which matters for IM-1 and IM-2 because both tipped over and the final resting place is not the pre-launch target.

Surveyor detail: [LPI Surveyor program](#).

CLPS planned missions: [NASA CLPS](#) for the official list and payload manifests. These sites move, so mark all of them `verify` and note the date you checked.

Outcomes and durations: the [NSSDCA master catalog](#) for historical missions. For the 2024–2026 landers you will be reading operator press releases.

Done means

Every row has a coordinate with a source URL beside it, no blanks in `lat_deg` or `lon_east_deg`, and three rows spot-checked on a lunar map to confirm they land where the mission is supposed to be.

Time: two solid days. Do not leave this to the week before.

Person 2 — Geometry and data pipeline

Your job in one line: turn Person 1's spreadsheet into `missions.json` by asking NASA's ephemeris service where the Sun and Earth were in the sky at each landing site.

The concept, since you will be explaining it

Stand on the Moon at a landing site and look up. The Sun is somewhere, Earth is somewhere. Each has an elevation angle: above 0° means visible, below means below the horizon. JPL Horizons will tell you both angles for any moment, past or future, at any point on the Moon. That is the whole calculation. We ask for it hourly across a year, then find the stretches where both are positive at once.

The six steps

Everything runs in Google Colab. No local setup.

1. **Learn the tool by hand before automating it.** Open [JPL Horizons](#). Set Ephemeris Type to Observer Table, Target to Sun, Observer Location to a coordinate on the Moon (body 301) using the Apollo 11 longitude and latitude, time span of a week at 1 hour steps, and tick **Apparent AZ & EL** under Table Settings. You get an elevation column. Watch it go positive and negative over about 29 days. That is a lunar day, and now you understand the data.
2. **Set up Colab.** New notebook, run `!pip install astroquery pandas numpy`, upload the five Python files.
3. **Load the data.** Paste Person 1's rows into `sites.py` following the format of the existing entries.
4. **Run it.** `!python run_precompute.py`. It queries Horizons per site, caches the results so re-runs are free, and writes `missions.json`.
5. **Read the sanity check output.** It tests that far-side sites report zero Earth-visible time, that near-side equatorial sites have a high duty cycle, and that ops time never exceeds sun time or Earth time. A failure is almost always a longitude sign error in Person 1's table. Fix the data, not the code.
6. **Eyeball the numbers yourself.** Apollo 11 should come out near 1.0 ops efficiency. Chang'e 4 should be exactly 0. If Apollo 11 reads 0.4, something is wrong and you should not ship it.

What else you own

A plain-language write-up of the method for Person 6's slides, plus the honest limitations.

The main one: Horizons treats the horizon as flat. Real terrain, crater rims and mountains are not modelled. At 85° south the Sun and Earth sit within a couple of degrees of the horizon, so terrain genuinely decides visibility there. **Our polar numbers are an upper bound.** Say that out loud in the pitch. Judges reward a team that names its own error source.

Resources

- [JPL Horizons web interface](#)
- [Horizons manual](#)
- [astroquery.jplhorizons docs](#)

Time: one day once Person 1 delivers. Do a dry run with five made-up sites this week so you are not learning the tool in November.

Person 4 — Map and mission panel

Your job in one line: a lunar map with 22 clickable markers, plus the detail panel that appears when you click one.

Do this first, this week

Confirm the lunar map tiles actually load in a browser. This is the most likely thing in the whole project to break, and you want to find out now rather than at 2am in November.

Make a plain HTML file, load Leaflet from a CDN, and point it at a NASA lunar tile layer. Start with [NASA Moon Trek](#), which publishes WMTS endpoints under its API and layer documentation. [USGS Astrogeology](#) has LRO LOLA and WAC global mosaics as a second option.

If tiles render, you are unblocked. If they are slow or the console shows a CORS error, tell the team immediately and prepare a fallback: one static lunar image used as a Leaflet `ImageOverlay`. Less impressive, works every time. Have it ready either way.

Two gotchas that will cost you an hour each

Leaflet assumes Earth by default: Web Mercator projection, Earth's radius. For lunar data you normally set `crs: L.CRS.EPSG4326` (plain lat/lon) instead. If your markers land in visibly wrong places while the tiles look fine, this is why.

Our data uses east-positive longitude, sometimes 0–360. Leaflet wants –180 to +180. Convert with `lon > 180 ? lon - 360 : lon`.

What to build

- Leaflet map with lunar tiles and sensible zoom limits
- One marker per mission, coloured by `metrics.ops_efficiency` from the JSON, so the pattern (Apollo equatorial, Chang'e far side, CLPS polar) is visible before anyone clicks anything
- Clicking a marker sets the app's selected mission; every other view listens to that
- A detail panel showing Person 1's mission content alongside Person 2's computed numbers

Resources

- [Leaflet tutorials](#) — do the quickstart and the custom-CRS one

- [NASA Moon Trek](#)

Time: a couple of hours this week for the tile test. The rest is weekend work.

Person 5 — Timeline, compare chart, polar plot

Your job in one line: three D3 charts that turn the JSON into the actual argument of the project.

Build in this order

1. **Compare chart. Build this first.** A scatter plot, one dot per mission. X axis is landing latitude, Y axis is ops efficiency. Colour by era or provider. Far-side missions sit flat on zero. This is the most valuable screen in the app and the cheapest to make, roughly 60 lines of D3. If the weekend goes badly and this is all that gets finished, the project still works.
2. **Timeline.** Horizontal bars along a time axis, three stacked rows per mission: sun-up intervals, Earth-up intervals, and the overlap band. The JSON gives these directly as `[start, end]` pairs in hours, so each interval is literally one `<rect>`. For missions that flew, mark how long the lander actually survived against how long the geometry allowed. The gap between those two is the story.
3. **Polar plot.** A circular sky chart: compass direction around the edge, elevation from horizon at the rim to overhead at the centre. Draw the Sun's path and Earth's path over one lunation. At Apollo 11 both arc high overhead; at 85° south they crawl near the rim. The JSON has a `tracks` array for exactly this. Drop this one if you run short: it is the nicest view and the most expendable.

Practical notes

Do not try to learn all of D3. You need four things: `d3.scaleLinear`, `d3.scaleTime`, `d3.axisBottom` / `axisLeft`, and appending SVG elements from bound data. Skip the layout modules, force simulations and everything else.

Time in the JSON is hours since the analysis window start. Convert with:

```
new Date(Date.parse(meta.analysis_window.start) + hours * 3600e3)
```

Start now with mock data. Person 2 freezes the schema; you hand-write a JSON matching it with three fake missions and build against that. Swap the file when real data lands. Do not wait for real data to start.

Resources

- [Observable D3 gallery](#) — find a scatter plot and a Gantt chart, read the code
- [D3 in Depth: scales and axes](#)

Time: compare chart a couple of hours, timeline half a day, polar plot half a day.

Deadlines and dependency order

Only two people have hard pre-event deadlines. If they deliver, the hackathon weekend is spent building and polishing. If they do not, the whole team spends Saturday doing data entry and we demo nothing.

Person	Pre-event task	Deadline	Blocks
1	Verified coordinates + mission facts for 22 sites	Early November	Person 2, and through them everyone
2	Run precompute, validate, freeze <code>missions.json</code>	Early November	Persons 4 and 5
4	Confirm lunar tile source loads in a browser	This week	Nothing, but a late failure is expensive
5	Hand-written mock JSON, build compare chart against it	Optional, recommended	Nothing

The dependency chain is simply Person 1 to Person 2 to the frozen schema, and once the schema is frozen Persons 4 and 5 can work in parallel without talking to each other. That is the point of freezing it.

Set Person 1 and Person 2's deadline at **early November**, not the week of the event.